Minimum Cost of String Transformation

Company: Google
Round: Online
Year: 2022
Difficulty: Hard
Topic: String, Greedy, Dynamic Programming, ASCII
Source: https://www.desiqna.in/8942/tcs-coding-questions-answers-october-2022-minimum-operation

Problem Description

Given a string S of length N containing lowercase English letters.
We can perform two operations:

Input Format

- The first line contains an integer N, representing the length of the string.
- The second line contains the string S.
- The string contains only lowercase English letters.

Output Format

Print a single integer representing the minimum possible value of X + Y.

Sample Input

4
Abcd

Sample Output

6

Explanation

The goal is to minimize X + Y:

- X = cost of replacing characters.
- Y = number of pairs where i < j and S[i] < S[j].
- We can change characters to reduce Y, but changes increase X.
- We need to find the best changes so that X + Y is minimum.

Approach to Solve This Problem:

1. Start with the original string.
2. Calculate the number of pairs (i, j) where i < j and S[i] < S[j]. This gives Y.
3. We can replace characters to reduce the number of such pairs.
4. Every replacement has a cost based on the ASCII difference between the old and new characters.
5. For each possible character change, consider:
    - Cost of changing the character → X
    - Number of increasing pairs after the change → Y
6. Choose the changes that give the minimum value of:

X + Y

Java Solution:

```java
import java.util.*;
public class Main {
    static String s;
    static int n;
    static long answer = Long.MAX_VALUE;
    static void solve(int index, char[] current, long cost) {
        if (cost >= answer) {
            return;
        }
        if (index == n) {
            long y = 0;
            for (int i = 0; i < n; i++) {
                for (int j = i + 1; j < n; j++) {
                    if (current[i] < current[j]) {
                        y++;
                    }
                }
            }
            answer = Math.min(answer, cost + y);
            return;
        }
        char original = s.charAt(index);
        for (char c = 'a'; c <= 'z'; c++) {
            long replaceCost = Math.abs(original - c);
            solve(index + 1, current, cost + replaceCost);
            current[index] = c;
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        s = sc.next();
        char[] current = new char[n];
        solve(0, current, 0);
        System.out.println(answer);
        sc.close();
    }
}
```

Solution:

The main idea is to balance the cost of changing characters (X) with the number of increasing pairs (Y).

1. Start with the original string and calculate the number of increasing pairs.
2. Consider changing a character to another lowercase letter.
3. Changing a character from a to b adds a cost of |a-b| to X.
4. At the same time, the change can increase or decrease the number of increasing pairs Y.
5. For every possible character choice, calculate the best possible value of:
   X + Y

6. Use dynamic programming to keep the minimum cost for each possible last character instead of trying every possible transformed string.
7. The minimum value among the final DP states is the answer.

Complexity Analysis:

Time Complexity: $O(N \times 26^2)$

Approximately O(N), since there are only 26 lowercase letters.

Space Complexity: O(26)

Approximately O(1), if only the previous DP state is stored.